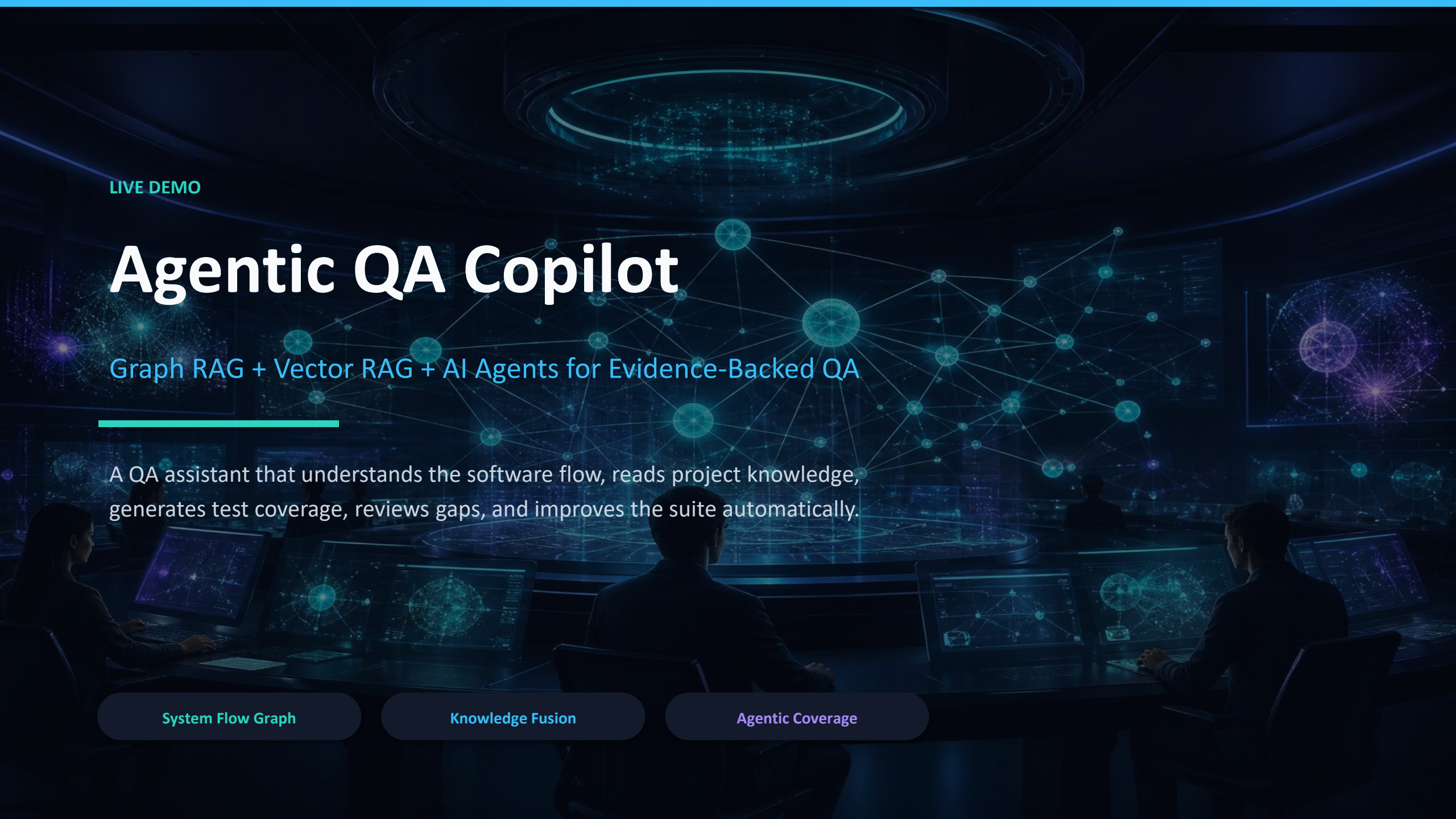


LIVE DEMO

Agentic QA Copilot

Graph RAG + Vector RAG + AI Agents for Evidence-Backed QA

A QA assistant that understands the software flow, reads project knowledge, generates test coverage, reviews gaps, and improves the suite automatically.

System Flow Graph

Knowledge Fusion

Agentic Coverage

Why Traditional AI Test Generation Falls Short

01

Generic test cases

AI tools often generate broad tests without understanding the actual product flow.

02

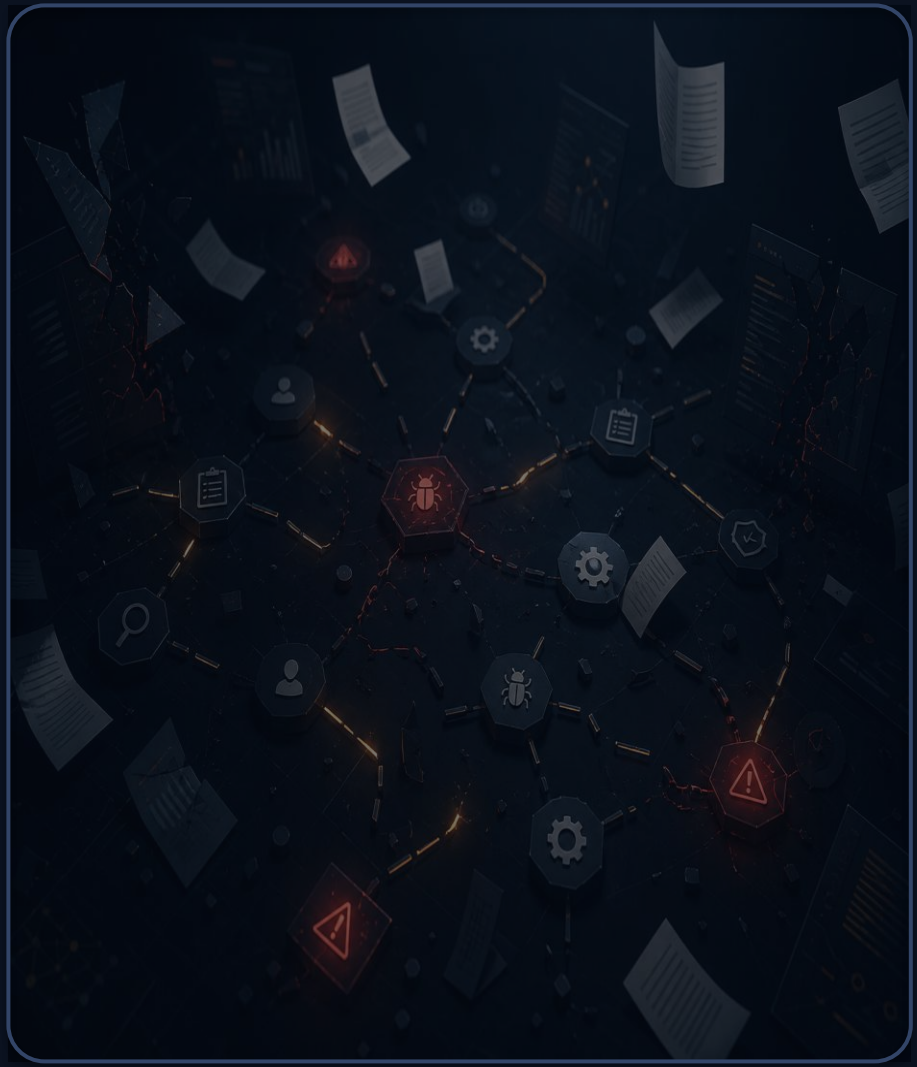
Missing business context

Requirements, bugs, risks, and existing tests are scattered across documents and tools.

03

Weak coverage visibility

QA teams struggle to see what is covered, what is missing, and what should be tested next.



How Our QA Copilot Understands the Product

System Flow Graph

Knowledge Base

Jira / Confluence

v

Graph RAG + Vector RAG

v

Context Fusion

v

QA Agents

- **Graph RAG**
  Workflows, branches, dependencies, failure paths
- **Vector RAG**
  Requirements, bugs, risks, and documents
- **Context Fusion**
  Combines both into high-quality prompt context
- **QA Agents**
  Generate, review, and refine QA outputs

From Input to Complete QA Output

1. Build system flow graph
2. Add knowledge / import Jira & Confluence
3. Run agentic analysis
4. Generate test cases
5. Review validity
6. Classify automation feasibility
7. Detect coverage gaps
8. Generate targeted tests
9. Export BDD / Gherkin

Agent Loop · Generate -> Review -> Fix -> Cover -> Export

OUTPUTS

- Test Cases
- BDD Feature Files
- Automation Review
- Exploratory Missions
- Bug Reports
- Regression Recs
- Coverage Gaps
- Sources & Evidence
- Agent Trace

What Makes This Demo Powerful

01

Context-aware QA

Uses actual system flow and project knowledge.

02

Evidence-backed results

Every output links to graph paths, requirements, bugs, or documents.

03

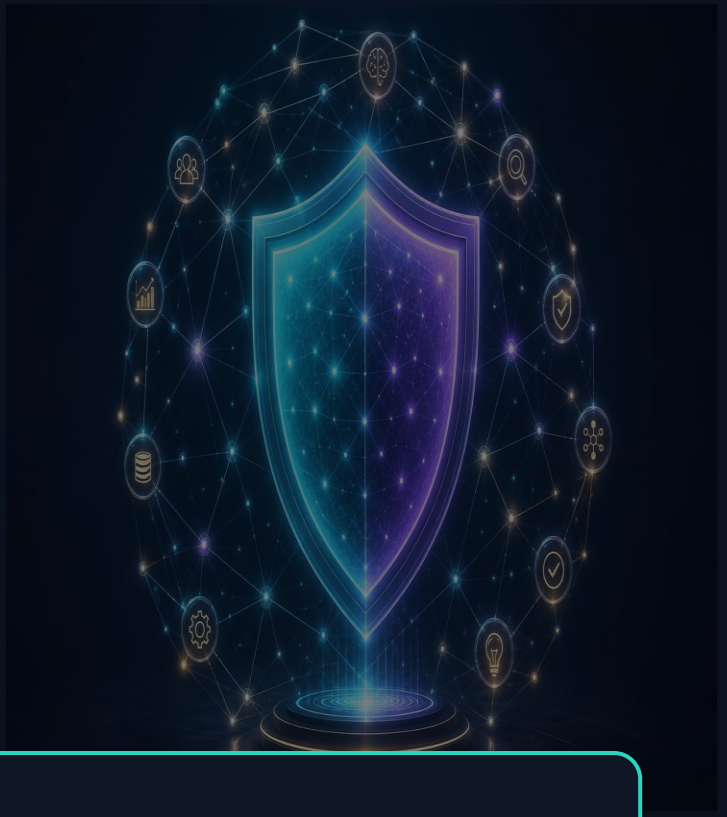
Closed-loop coverage

Reviewer finds weak or missing tests and sends them back for refinement.

04

Automation-ready planning

Classifies tests as automatable, manual, hybrid, or not ready.



LIVE DEMO FOCUS

System Flow -> Knowledge Import -> Agentic Analysis -> Results -> BDD Export

Future Scope

From QA assistant to continuous AI QA platform

1

Deeper Jira + Confluence

Auto-sync epics, stories, acceptance criteria, and product docs.

2

Test Execution Integration

Connect Playwright, Selenium, Cypress, Postman, and CI/CD.

3

Automated Script Generation

Convert approved BDD into runnable automation code.

4

Release Readiness Dashboard

Risk, coverage, automation readiness, and open QA gaps.

5

Continuous QA Monitoring

Re-run analysis when requirements, tickets, or flows change.

6

Enterprise Governance

RBAC, audit logs, approval workflows, reusable QA templates.

Limitations & Current Scope

What the current demo proves - and what remains for production hardening

The demo validates the core agentic QA workflow, while some production capabilities require further hardening.

01 Input quality matters

QA output quality depends on the completeness of the system-flow graph and uploaded knowledge.

02 Coverage is modeled coverage

Coverage is measured against modeled graph paths, requirements, risks, and evidence - not infinite real-world possibilities.

03 Human QA approval is still required

Candidate bugs, generated tests, and automation recommendations should be reviewed by QA experts before release use.

04 Integration maturity

Jira, Confluence, execution tools, and CI/CD need production-grade auth, sync, permissions, and monitoring.

05 Runtime and cost governance

LLM usage requires model routing, rate-limit handling, fallbacks, observability, and cost controls.

Current demo = context-aware QA intelligence.
Next step = continuous enterprise QA platform.